

Mega2 Development Notes

June 15, 2016

Robert V. Baron

Department of Human Genetics, University of Pittsburgh

Copyright © 2012-2014 by the University of Pittsburgh

Contents

1	Defining an Analysis Method	2
1.1	Class definition	2
1.2	Class code	3
1.3	Glue code	3
2	Generating Output Files	4
2.1	Simple Output Classes	4
2.2	Output Loop Classes	5
3	Legacy Structures	13
4	Compilation Tricks	13
4.1	MinGW	13
4.2	Microsoft Windows	13
5	Analysis Header	14
5.1	loops()	14
5.2	Allow_no_traits()/Allow_no_chr	14
5.3	Allele_data_use_name_if_avaialable	15
5.4	Forbid_sex_linked_loci	15
5.5	Ped_ind_defaults(int unique)	15
5.6	Batch_In, Batch_Out, Batch_Show	15

1 Defining an Analysis Method

To make this discussion concrete we will look at the implementation of the FBAT analysis method. First, we need to choose two file names for the analysis method: one for the C++ code and one for the class definition. We will use the files **write_fbat.cpp** and **write_fbat_ext.h**, respectively. (Technically, there can be executable code in the class definition.)

1.1 Class definition

We define a new class and derive it from the “**CLASS_ANALYSIS**” base class. (I like to put the string “CLASS_” at the start of the class name.) After defining the constructor/destructor there are a lot of standard methods that can be defined and other methods that will be forward referenced and defined in the “code” file. And of course you can define any methods you need to define the semantics of the analysis.

All these standard methods must be a subset of what is defined in **CLASS_ANALYSIS** (see file *analysis.h*). In particular, a few dozen virtual methods are defined that contain only a “return false” statement. There are extensive comments in the *analysis.h* file that document what these switches do. These virtual methods turn off or on features of the Mega2 system necessary for this particular analysis method being defined. The base class defines these switches to be off. Thus if that is the correct state for the new analysis, it is not necessary to repeat the method definition since it will be inherit from the base class. Of course, the method is virtual so you may define it to “return true” for this analysis. Obviously, in this case it would be necessary to copy the appropriate definition into the new class definition.

Toward the end of the **CLASS_ANALYSIS** definition, there are a few more virtual methods that you might want to override for your class: *ped_ind_defaults*, *sub_prog_name*, *sub_prog_name_to_sub_option*, and *interactive_sub_prog_name_to_sub_option*. Often, certainly for FBAT, you only need to define *ped_ind_defaults*, which indicates how to print out the pedigree and person information. For the other functions, the default is fine.

Finally, the four methods: *create_output_file*, *get_file_names*, *file_names*, and *replace_chr_number* are declared with the class definition but defined in the code file. *create_output_file* is responsible for doing data transformations and creating all the output files needed by the analysis program. The other methods support the idiosyncratic way that Mega2 generates file names (with embedded chromosomes and suffix that indicates file type).

1.2 Class code

The header of the file is standard boiler plate and you should just copy it from an existing .cpp, such as **write_fbat.cpp**. You will of course not include the “write_fbat_ext.h” (and “fbat.R.h”) header, but use the header file for your class. I like to follow the header by the code to generate each of the types of files needed by the analysis and at the end of the file I define the *create_output_file* to call all the methods (or functions) defined earlier in the file. Finally, at the end of the file there is the code to interactively query the user for the names to be used for each type of file, *get_file_names*. And there is the code to glue the names together and to allow components (like the chromosome number) to be changed as necessary.

So lets look deeper into how an output file is generated. But first we need to make a few adjustments to the Mega2 files so these new files are compiled and available.

1.3 Glue code

1. Towards the head of analysis_ext.h, you should add: include "write_fbat_ext.h"
2. Towards the end of analysis_ext.h, you should export the new class (I named it CLASS_FBAT):

```
extern    CLASS_FBAT *FBAT;
```

3. Towards the head of analysis.cpp, before analysis_list[] is defined, you should instantiate FBAT

```
CLASS_FBAT *FBAT = new CLASS_FBAT();
```

4. Towards the head of analysis.cpp, in the analysis_list[] definition, you should add FBAT

```
{ "FBAT format", FBAT },
```

5. In prog_name_to_number(), add a case to recognize the name "fbat" and return the object FBAT
6. Add write_fbat.cpp to the Makefile SRCS variable
7. Execute: make depend

2 Generating Output Files

2.1 Simple Output Classes

class file_ops [in file_ops.h]

“file_ops” opens [*filep_open()*] and closes [*filep_close()*] files as well as prints [*pr_printf()*, *pr_puts()*, and *pr_nl()*] to files. If the file name ends in “.gz”, the compression library, zlib, will be used to write the file data. The print methods use the file descriptor member of the object and take the same arguments as printf and fputs, respectively. The *filep_open* method calls the virtual function, *file_header*, after the file is opened; while the method *filep_close()* calls the virtual function, *file_trailer*, before the file is closed and the virtual function, *file_post*, after it is closed. These virtual functions allow derived classes to easily supply headers and trailer data for the file.

In addition, there are additional methods to flush the data to disk, *file_flush*, and delete the named file, *filep_delete*.

Note: The object destructor calls the *filep_close* method; so it is never necessary to explicitly call *filep_close*.

class person_locus_entry : public file_ops [in entry.h and entry.cpp]

This class contains all the necessary members to identify a pedigree/person, locus and alleles. It contains auxiliary members to find the additional properties of the person and locus. And it holds other bookkeeping data. This class is derived from “file_ops” to use the latter’s printing capabilities. This class has numerous methods, typically beginning “pr_”, for printing: person, parents, sex, phenotypes, alleles, etc. If it proves necessary to access the members without printing their data, the members are defined as:

member	type	meaning
_ped	int	pedigree index
_per	int	person index
_locus	int	locus index
_allele1	int	first allele value as integer
_allele2	int	second allele value as integer
_trait	int	trait index (in locus array)
_numchr	int	chromosome number
_chrom_loop	int	1 means produce a separate file per chromosome
_Top	linkage_ped_top *	base for everything
_LTop	(linkage_locus_top *	base for locus array
_EXLTop	ext_linkage_locus_top	base for extended locus array
_tpe	linkage_ped_rec *	info for person
_tp	linkage_ped_tree *	info for pedigree
_tle	linkage_locus_rec *	info for marker locus
_tte	linkage_locus_rec *	info for trait locus

Note: All these member names begin with an underscore “_”.

2.2 Output Loop Classes

The loop classes are defined in the files: **loop.h** and **loop.cpp**. These supply an Inner loop and Outer loop. Both loops contain several different classes each of which iterates through people, chromosomes and markers in a designated order. Creating an output file for Mega2 is achieved by creating a new class that is derived from a chosen Output loop class and a chosen Inner loop class. It is easiest to explain the Inner loop and then the Outer loop but programmatically you need to setup the Outer loop and then the Inner loop. So it might be best to read this section twice.

Note: (This will also make more sense on a second reading.) Both the Inner and Outer loop are derived from **person_locus_entry** class. When we create the output file class, it will be derived from both an outer class and an inner class. This is called multiple inheritance. And each of these classes is derived from the person_locus_entry class. This is called OOPS. Although this could lead us into a long discussion about inheritance, the basic question is do we want the inner loop object to inherit from a different object than the outer loop object. This would be the C++ default. But we want both objects to inherit from the same person_locus_entry object and share data. To have this happen correctly, the definition of each type of loop class must indicate that the class is derived from the “virtual” person_locus_entry class. After this accommodation, there is only one more small detail that must be attended to in creating the class for the output file.

Inner loop

A class of the Inner loop type will create an output file. Let us start by explaining, in detail, one example: the `loop::ped_per_loci` class. There is only one “real” method, *run_loop()*, in the class, but quite a few virtual function that are stubbed out as empty to be replaced in a derived class as necessary. The goal of this class is to open and output file and then iterate over all the pedigrees and then in each pedigree iterate over each person and then for each person iterate over all the markers in some chromosome. The file is open on line 8. Line 11 starts the loop on pedigrees, and line 20 starts the loop on loci on the specified chromosome. Line 11 sets and increments `_ped`, which is used on line 17 to define `_tp`, a pointer to the pedigree info. A similar arrangements happens on line 20/23 also on 27/29/30. Again these constructs define variables that can be accessed, but by what? The interesting part of the loop is lines 18/46, 25/44, and 41 which invoke the virtual methods *ped_start/ped_end*, *per_start/per_end* and *inner*, respectively. The `ped_per_loci` class has these functions do nothing, but a derived class can override the methods as needed. For example, if you needed to relabel the pedigree (and or persons), that code could be placed in the *ped_start* method. If you needed to print the person id, this could be done in the *per_start* method. This class could be used to list the genotypes for each person. In this case, the *inner* method would print the genotypes. It could also print a new line or the new line could be deferred to the *per_end* method, thus producing a long line starting with the person id followed by all the allele labels. (Note: if all this is clear, you should see that the first option will only produce the person id on the first line per pedigree/person).

For completeness, here are a few more details: line 6 works magic to fill `NumChrLoci` (line 27) with the number of traits and markers for the specified chromosome and fill the `ChrLoci` Array (line 29) with the loci index for the traits and markers for the specified chromosome. There are checks to make determine if untyped pedigrees should be processed (line 14), only markers are processed (line 34) and that there be less than a specified numbers of alleles for the marker (line 36).

```
1  \ \ void loop::ped_per_loci::run_loop(const char *dir , const char *fl_name ,
2  \ \ {
3  \ \     int m;
4  \ \
5  \ \     // Loop through the Loci ...
6  \ \     markers_on_chromosome(_numchr);
7  \ \
8  \ \     filep_open(dir , fl_name , mode);
9  \ \
10 \ \     // for each pedigree (consulting the linkage_ped_top structure) ...
11 \ \     for (_ped=0; _ped < _Top->PedCnt; _ped++) {
12 \ \
```

```

13  \ \          // It there is some indication as to why this pedigree should n
14  \ \          if (UntypedPeds != NULL && UntypedPeds[_ped]) continue;
15  \ \          // for each person/individual in that pedigree (cousulting the l
16  \ \
17  \ \          _tp = &(_Top->Ped[_ped]);
18  \ \          ped_start();
19  \ \
20  \ \          for (_per = 0; _per < _Top->Ped[_ped].EntryCnt; _per++) {
21  \ \              // record for the individual (a linkage_ped_rec)...
22  \ \
23  \ \              _tpe = &(_Top->Ped[_ped].Entry[_per]);
24  \ \
25  \ \              per_start();
26  \ \
27  \ \              for (m=0; m < NumChrLoci; m++) {
28  \ \                  // The actual Loci number associated with a SELECTED Lo
29  \ \                  _locus = ChrLoci[m];
30  \ \                  _tle = &(_LTop->Locus[_locus]);
31  \ \
32  \ \                  // Ignore anything but a marker.
33  \ \                  // This should get us things of Locus[?].Type == {NUMBE
34  \ \                  if (_tle->Class == MARKER) {
35  \ \
36  \ \                      if (_loci_allele_limit && _tle->AlleleCnt > _loci_
37  \ \
38  \ \                      // The markers for this individual...
39  \ \                      get_2alleles(_tpe->Marker, _locus, &_allele1, &_all
40  \ \
41  \ \                      inner();
42  \ \                  }
43  \ \              }
44  \ \              per_end();
45  \ \          }
46  \ \          ped_end();
47  \ \      }
48  \ \      filep_close();
49  \ \ }

```

There is another Inner loop class that loops first over the loci then the person, loci_ped_per. The *inner* method is applied every time the person changes. Also, the outer most iteration

uses *loci_start/loci_end*. Similarly, there is a pair of Inner loop classes with trait instead of loci. Filling out the grouping there is a loop class for loci, and one for pedigree/person and one that is not a loop at all ... that happens just once.

Outer loop

Mega2 sometimes outputs one file of data. But it can also output separate files in new directory for each different trait and having a different name (suffix) for different chromosomes. The example shown above processes one chromosome. (Although if the `_numchr` value is -1, it will process all chromosomes into one output file.) How do we iterate through all the chromosomes (and/or possibly traits)? This is what the Outer loop classes support.

Thus building on the example above, supposed we want to iterate on chromosomes. The Outer loop class, ***chr***, handles this case. There is only one “real” method, *iterate()*, in the class, but quite a few virtual function that are stubbed out as empty to be replaced in a derived class as necessary. (All this is analogous to the Inner loop class described above.) The goal of this class is to define the file that will be used by the Inner loop and then iterate over all the chromosomes. Or if chosen by the user, not iterate at all and process all the chromosomes together. Line 22 starts a loop; but if `_chrom_loop` is zero, the loop will be degenerate, and only go through one cycle and abort at line 43. The code in lines 24-35 assure that the correct chromosomes for will be iterated through. If `_chrom_loop` is set choose the correct chromosome (line 25) for each cycle; otherwise choose all the chromosomes (line 29) or the only single chromosome (line 32). The interesting part of this loop construct is lines 37/42 which invoke the virtual methods *chr_start/chr_end*. These are intended to be overwritten as necessary similar to the start/end virtual methods defined earlier. Finally, line 39, *make_file()*, is the glue that will connect the Outer loop to the Inner loop. In particular, the “new” class discussed earlier must define a *make_file* method that calls the *run_loop* method.

For completeness, there is some “magic” code lines 6-20. This code chooses the first trait of a given type (Affection status or Quantitative) to make available to all the subsequent running code. It exists to satisfy usages that need to define a trait even though we are not looping over traits.

```
1  \ \ void loop::chr::iterate()
2  \ \
3  \ \     int i, tr;
4  \ \     linkage_locus_rec *tte;
5  \ \
6  \ \     _tte = (linkage_locus_rec *)NULL; // None was selected
7  \ \     for (tr=0; tr < num_traits; tr++) {
8  \ \
```



```

9  \ \      _trait = *_trp++;
10 \ \      if (_trait == -1) continue;
11 \ \      if (_trait == -99) break;
12 \ \
13 \ \      tte = &(_LTop->Locus[_trait]);
14 \ \      if (_trait_affect && tte->Type != AFFECTION)
15 \ \          continue;
16 \ \      if (_trait_quant && tte->Type != QUANT)
17 \ \          continue;
18 \ \      _tte = tte;
19 \ \      break;
20 \ \    }
21 \ \
22 \ \    for (i = 0; i < main_chromocnt; i++) {
23 \ \
24 \ \        if (_chrom_loop) {
25 \ \            _numchr=global_chromo_entries[i];
26 \ \            _Top->analysis->replace_chr_number(file_names, _numchr);
27 \ \        } else {
28 \ \            if (main_chromocnt > 1) {
29 \ \                _numchr=-1;
30 \ \                _Top->analysis->replace_chr_number(file_names, 0);
31 \ \            } else {
32 \ \                _numchr=global_chromo_entries[i];
33 \ \                _Top->analysis->replace_chr_number(file_names, _numchr)
34 \ \            }
35 \ \        }
36 \ \
37 \ \        chr_start();
38 \ \
39 \ \        make_file(); // which must call run_loop() to access an 'inner'
40 \ \
41 \ \        chr_end();
42 \ \
43 \ \        if (!_chrom_loop) break;
44 \ \
45 \ \    }
46 \ \ }

```

There is another Outer loop class, trait, that loops over traits in a similar manner to the

chromosome loop; this loop exposes *trait_start/trait_end* instead of the “chr” methods. Another is a loop iterates both over chromosomes and with each chromosome value, iterates over traits, exposing the virtual methods *chr_start/chr_end* as seen earlier as well as *trait_start/trait_end*. Finally, there is a loop that is not a loop at all ... that happens just once.

An important observation about all the Outer loop classes, is that their constructors examine that global variables set for the analysis and then define *_chrom_loop* and *_trait_loop* (and a few others) to carry into the loop semantics whether the user desires “looping” for the traits (i.e. creating separate directories) and for the chromosomes (i.e. creating separate files).

One final note: it seems that there might be more Outer loop (and Inner loop) classes than are necessary. The answer is that the flexibility is necessary. For example, you might use the Outer loop over chromosome and trait class to produce a file that contains the person id and all his/her phenotypes. That would create a file in each trait directory (possibly labeled by the chromosome.) But all these files would contain the same data. In this case, using the “not a loop” case is what is probably wanted. Of course, if the analysis needs a control file for each trait and chromosome, then the first choice was correct. And if the control file has some lines that depend on the trait/chromosome and others that the user would set depending on the “subanalysis”, you might use both loops and when the control file with the trait/chromosome information is written add a line to include the “subanalysis” control file.

Combining the Outer loop class and the Inner loop class

So pulling this all together, suppose we want to have the Outer loop encompass the trait and chromosome and the Inner loop handle pedigree/person/marker. (Think a ped file.) We would start by defining a class that inherits from both:

```
struct new_class: public loop::outer, loop::ped_per_loci {
}
```

Its constructor would look a little strange:

```
new_class (linkage_ped_top *Top) : person_locus_entry(Top), loop::outer(Top), loop::
```

We indicated earlier that the loop constructs are strange because they all inherit “virtually” from *person_locus_entry*, i.e. multiple objects share the same *person_locus_entry* object. The result is that we have to explicitly add the virtual base class constructor to the list of base class constructors to invoke before calling the new constructor.

Finally, we need to define

```
make_file()
```

to print out a file name to show the user what is happening and then to call

```
run_loop(<file name to use>);
```

Example The code snipped below comes from the **write_fbat.cpp** file: I am not going to explain everything, just the highlights. The code illustrates the intended use of the virtual methods allowed for in the loop classes. *file_header* emits the name of each marker at the beginning of the file. *per_start* prints the person id and parentage, etc. *per_end* simply prints a newline. And of course, *inner* prints the marker name.

One more idiosyncrasy of the example code needs to be explained. Lines 5-44 define the **save_peds** class and create a new **save_peds** object pointed to by sp. Line 54 calls the *iterate* method. So what happens on lines 46-52? Lines 46&47 copy data that is available to the outer function into the object (also see lines 7& 8.) Some version of C++ have lexical scoping so this is not necessary, but not all; so this is the safe way to go. Lines 49 & 50 initialize some parameters of the loop: that traits should only be affection status and that there may be at most 40 alleles per marker. Finally, line 52 carries some formatting information into the loop that will be used to print family ids and person ids.

```
1  \ \ static void save_FBAT_peds(linkage_ped_top *Top, char *file_names [],
2  \ \                                     const int pwid, const int fwid,
3  \ \                                     const bool has_x)
4  \ \ {
5  \ \     struct save_peds: public loop::outer, loop::ped_per_loci {
6  \ \         typedef char *str;
7  \ \         str *file_names;
8  \ \         bool has_x;
9  \ \
10 \ \         save_peds(linkage_ped_top *Top) : person_locus_entry(Top), loop
11 \ \         void make_file() {
12 \ \             mssgvf("          FBAT pedigree file:    %s/%s\n", *_opath, f
13 \ \             if (_tte == NULL) {
14 \ \                 mssgvf("          FBAT pedigree file:    needs a trait va
15 \ \                 EXIT(DATA_INCONSISTENCY);
16 \ \             }
17 \ \             run_loop(file_names[0]);
18 \ \         }
19 \ \         void file_header() {
20 \ \             int locus;
```

```

21  \\\           int m;
22  \\\           linkage_locus_rec *lpe;
23  \\\           if (has_x) return;
24  \\\           for (m=0; m < NumChrLoci; m++) {
25  \\\               locus = ChrLoci[m];
26  \\\               lpe = &(_LTop->Locus[locus]);
27  \\\               if (lpe->Class == MARKER)
28  \\\                   pr_printf("%s ", _Top->LocusTop->Locus[locus].Name)
29  \\\           }
30  \\\           pr_nl();
31  \\\       }
32  \\\       void per_start() {
33  \\\           pr_id();
34  \\\           pr_parent();
35  \\\           pr_sex();
36  \\\           pr_aff();
37  \\\       }
38  \\\       void per_end() {
39  \\\           pr_nl();
40  \\\       }
41  \\\       void inner() {
42  \\\           pr_marker();
43  \\\       }
44  \\\   } *sp = new save_peds(Top);
45  \\\
46  \\\   sp->file_names = file_names;
47  \\\   sp->has_x = has_x;
48  \\\
49  \\\   sp->_trait_affect = true;
50  \\\   sp->_loci_allele_limit = 40;
51  \\\
52  \\\   sp->load_formats(fwid, pwid, -1);
53  \\\
54  \\\   sp->iterate();
55  \\\
56  \\\   delete sp;
57  \\\ }

```

3 Legacy Structures

4 Compilation Tricks

4.1 MinGW

MinGW is described in www.mingw.org. The download can be found at <http://www.mingw.org/download/installer>. Once downloaded, the binaries should be added to your Windows path. At this point, a build of Mega2 should proceed just like a regular Unix/Linux/-MacOS build. You can either type

```
make
```

in the Mega2/srcdir directory or type

```
install
```

in the Mega2 directory. The binary that is produced depends on a two .dll's that are part of the MinGW libraries; these .dll's must be packaged with the Mega2 binary. The .dlls can be found in:

.dll	path
libgcc_s_dw2-1.dll	XXXXXX
libstdc++-6.dll	XXXXXX

4.2 Microsoft Windows

Visual Studio

First, you should obtain a copy of the latest Visual Studio or Visual Studio Express from Microsoft www.visualstudio.com. If you are given a choice, only the C and C++ compilers tool chain is necessary.

Gmake

You will also need a copy of Gnu make <http://www.gnu.org/software/make/> for windows to run the make scripts that build Mega2. A windows binary can be found at <http://gnuwin32.sourceforge.net/packages/make.htm>.

Path

You must put the compiler and make binaries on your path. The easiest way to put the compiler and tools onto your path is to “source” “c:\Program Files\Microsoft\Microsoft Visual Studio\Common\bin\vs32vars.bat XXXXX”. A simple

```
PATH %PATH%;<full path to make binary>
```

will put the make program in your path.

NOTE: These two procedures have to be repeated every time you need to compile Mega2.

Execution

With these prerequisites fulfilled, you need to enter the *Mega2/srcdir* directory and type

```
make
```

to compile Mega2. The binary that is produced is self contained and will run on any windows system.

5 Analysis Header

Sometimes while working on developing a new analysis mode you may think of a feature or ability you’d like to have. Before going and building the wheel yourself you should check out some of the analysis.h file and see what functions you can inherit to make life easier.

Some are simple such as loops() which simply enables you to run loops as discussed above. Some are more complicated and I’ll try to explain some of them here.

5.1 loops()

This is probably the most important function to have for your analysis mode. It allows for looping in general.

5.2 Allow_no_traits()/Allow_no_chr

You can use this if you don’t want the ability to break up your analysis into different trait folders. Useful if an analysis type doesn’t do anything with traits.

Also you can allow no chromosome information if an analysis is able to be run without that information.

5.3 Allele_data_use_name_if_avaialable

Suppose you need to force the use of the Letter names of the Alleles, this requires mega2 to output with them.

5.4 Forbid_sex_linked_loci

If your analysis mode is unable to work on the the X, Y and MT chromosomes you will want to set this to true.

5.5 Ped_ind_defaults(int unique)

This is an interesting function because it defines how person and family ids are coded by Mega2 for your analysis mode.

There are two values in an array as part of this function, OrigIds[0] and OrigIds[1], the 0th entry refers to Pedigree ID choices, the 1st entry refers to person ID choices.

These effect the way pr_fam and pr_per behave in the entry.cpp file. You can set both OrigIds to int to correspond to different ways of printing IDs according to whatever you need to output.

Pedigree ID choices:

- 1=Pedigree field for post-makeped format
- 2=Pedigree field for pre-makeped
- 3=Consecutive numerical ids for nuclear pedigrees
- 6=Original pedigree file id pulled from the input file.

Individual ID choices:

- 1= Person field for post- or pre-makeped files
- 2 = Unique Ids
- 5 = Consecutively numbered person ids
- 3,4 = not used, although these are handled.
- 6=The person id pulled in from the input file.

5.6 Batch_In, Batch_Out, Batch_Show

These are options you can add to your analysis mode if you ask additional queries of the user. The idea is that you can load in values from MEGA2.BATCH and use them as your user inputs.

You can define these functions in your analysis mode and then call them at the appropriate time to get information you need or to output information for subsequent runs.

Inside of `Batch_Input.h` there are a variety of functions defined to write and read values to and from the Batch file. You can use these for your new batch fields.

`Batch_Show` is simply used to display batch values that have been loaded in.

6 Debugging Tricks

You'll begin by making sure you have `lldb` installed. You can check if it's already on your machine by typing.

```
whereis lldb
```

If it's not installed you should go and install by following the instructions at <http://lldb.llvm.org/build.htm>

Next you want to go back to your **Mega2/srcdir** now instead type:

```
make dgb
```

This calls the flags appropriately so that you can begin debugging. From there you can use either `lldb` directly in the command line or use `emacs`. To use `lldb` you can either set breakpoints and step through as normal or use a print statement like the following:

```
asm("int $3");
```

This command forces a breakpoint. From here you can debug as normal using `lldb`.

A current goal is to use `gdb` (or `lldb`) to debug on Eclipse, so far this has not been accomplished.